ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/331374199

A Machine Learning-Driven Approach for Proactive Decision Making in Adaptive Architectures

Conference Paper · March 2019

DOI: 10.1109/ICSA-C.2019.00050

CITATIONS
2

READS
319

2 authors:

Henry Muccini
Università degli Studi dell'Aquila
189 PUBLICATIONS **2,275** CITATIONS

SEE PROFILE

Karthik Vaidhyanathan
Gran Sasso Science Institute
6 PUBLICATIONS **2** CITATIONS

SEE PROFILE

Some of the authors of this publication are also working on these related projects:

Project ICT Living lab @ DISIM-UnivAQ View project

Project Collaborative MDSE View project

All content following this page was uploaded by Karthik Vaidhyanathan on 05 March 2019.

The user has requested enhancement of the downloaded file.

A Machine Learning-driven Approach for Proactive Decision Making in Adaptive Architectures

Henry Muccini
University of L'Aquila
L'Aquila, Italy
Email: henry.muccini@univaq.it

Karthik Vaidhyanathan
Gran Sasso Science Institute
L'Aquila, Italy
Email: karthik.vaidhyanathan@gssi.it

***Abstract*—Self-adaptation is nowadays considered to be the best solution to dynamically reconfigure a system in the occurrence of deviations from the expected quality of service (QoS) parameters. However, data- and event-driven systems, such as IoT applications, impose new heterogeneity, interoperability, and distribution issues, that make uncertainty on the QoS stability even harder. Typical adaption techniques make use of *reactive* approaches, an after-the-fact adaptation that starts when the system deviates from the expected QoS parameters. What we envision is instead a *proactive* approach to anticipate the changes before the event of a QoS deviation. More specifically, we propose IoTArchML, a machine learning-driven approach for decision making in aiding proactive architectural adaptation of IoT system. The approach i) continuously monitors the QoS parameters; ii) predicts, based on historical data, possible deviations from the acceptable QoS parameters; iii) considers a list of possible alternative solutions to prevent the QoS deviation; iv) selects the optimal solution from the list; and v) checks whether the envisioned solution satisfies the overall system QoS requirements. We, therefore, move the focus from self-adaptive architectures to *self-learning* architectures, enabling the architectures to learn and improve over a period of time.**

***Keywords*-Adaptive architectures, Machine learning, IoT architectures, Proactive decision making, Self-learnable**

I. Introduction

The field of self-adaptive systems has evolved over the years especially due to the increasing cost of handling software complexity [1]. The major reason for this trend is essentially due to the fact that the software components are becoming more and more heterogeneous and the goal/run-time requirements of the software keep changing with time. This can be clearly observed in the last decade with software becoming more pervasive in nature especially with the advent of the Internet of Things. IoT development is not just about software, but it is about a system containing software, as well as a set of connected components such as sensors, actuators, anything, and everything [2]. According to Gartner, 8.4 billion IoT devices were in use in 2017 and has predicted this value to increase to 21 billion by 2020 [3]. On one hand, IoT systems are becoming very popular and they are being widely adopted; whereas on another hand, developing and maintaining IoT systems poses a lot of challenges such as heterogeneity, interoperability, security etc. These challenges in turn lead to various QoS concerns [4] [2]. These include concerns like reducing data traffic, energy consumption, utilization, etc. In this direction, different self-adaptive solutions have been proposed in the literature. However, they are mostly *reactive* in nature, where the adaptation is performed to react to an arising issue. Encoding the different possible adaptation scenarios at design-time (to perform architectural adaptation at run-time) might not work on the highly dynamic IoT architectures. Machine learning techniques can be considered towards aiding adaptation as it ensures that the system *can learn from multiple scenarios and improve over a period of time*. This was suggested by [1]. However, what proposed is reactive in nature, since based on predictive rule-based adaptation methods or reinforcement learning based methods. In the case of existing prediction methods, the different rules need to be defined upfront; instead, in case of reactive methods using machine learning, verifying the quality of adaptation performed is a big challenge [1]. Hence, our goal in this work is to propose a *proactive* approach where *the architecture has the ability to learn, foresee the possible QoS issues (before they happen), generate the best adaptive strategy, verify the strategy and proactively adapt based on the data generated by the IoT system*.

The major contribution paper is threefold. First, the approach proactively identifies the need for adaptation of the IoT architecture even before an issue arises. It achieves this by continuously monitoring the QoS parameters as time-series data and uses machine learning models to forecast QoS values. Second, the approach uses the forecast to automatically identify the best adaptation strategy from a set of strategies by using machine learning techniques. This strategy verified by a model checker before executing the adaptation. Three, the approach further gathers the feedback of selected at time t, using the forecast at the time, $t + 1$ which is then used to further improve the decision making. Thus, ensuring that the architecture learns continuously from the feedback and improves the quality of adaptation eventually enabling the architecture to learn. Going by definition of machine learning [5], we define the learnability of a software architecture as the ability such that *"Given a task T of solving an architectural problem, given the architecture A of the IoT system, and a set of quality attributes Q that models the QoS of the system, the architecture A of the system improves with Data D, such that the QoS Q of the system is not compromised"*.

II. Related Work

Self-adaptive frameworks for IoT systems and Cyber-Physical Systems (CPS) have been presented. Model Driven Engineering (MDE) based approaches were presented in [6], [7] and [8] where the adaptation is carried out using the concept of models@run.time and MAPE-K loop. The idea of Emergent Configuration (EC)) for engineering IoT systems was proposed in [9] where the adaptation was carried out based on a goal-oriented approach. An agent-based framework for performing self-adaptation for IoT applications was proposed in [10]. The framework uses neural networks and evolutionary algorithms to carry out decision making for performing adaptation.

Some papers on the use of Machine Learning for self-adaptive systems in other domains have been presented as well. Reinforcement learning-based approaches for planning and adaptation in software systems were presented in [11] and [12]. An analysis of different decision-making techniques in dynamic self-adaptive systems was performed in [13]. [14] presented a multi-learner approach for self-adaptive and online QoS Modeling for cloud-based software services. A self-adaptive mechanism using machine learning and reinforcement learning for adaptation in autonomous systems was proposed in [15].

Differently, from the related works reported above, our approach identifies the need for adaptation at an early stage by proactively forecasting QoS values using machine learning models. This forecast is then used to drive the decision-making process, which uses another machine learning model (as compared to the traditional rule-based models) to determine the optimal adaptation strategy. The selected strategy is verified using a model checker and the feedback of the strategy selected is further fed back to the model to improve the decision-making process. This allows the approach to continuously evolve the machine learning model using the knowledge gathered in the form of feedback. The proactive nature of the approach also ensures that any wrong choice, made by the decision-making process, is identified immediately in the next step, thus preventing any adverse effects on the system.

III. Motivating Example

Consider a smart room system which provides two functionalities. The first one consists in granting or denying the access to a room based on the number of people inside the room and its capacity. The second one consists of the automatic opening and closing of the windows based on the oxygen levels inside the room. Figure 1 shows the high-level architecture of the system. It basically consists of four components: a `people counter` to count the number of people entering and exiting a room, the `oxygen sensor` for reading the oxygen levels, the `controller` for piloting the actuators when opening and closing the doors and windows, and a `database` for receiving and storing the data.

The `oxygen sensor` may operate according to two operational modes: the *critical mode* (which gets activated when the level of oxygen in the room goes below a threshold value; during this mode it sends data to the controller every 1 second) and in the *normal monitoring mode* (where it sends data to the controller every 10 seconds). The `people counter` keeps sending the data to the controller as and when a person enters the room. The `controller` component, in turn, takes the decision on the opening and closing of the doors and windows and keeps sending the data collected from the sensor and counter to the `database` component every 5 seconds.

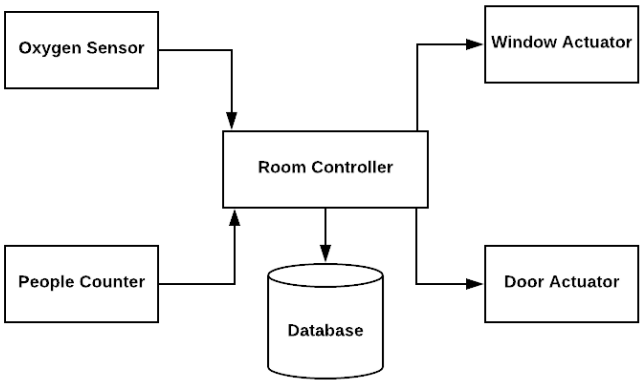

Fig. 1. Smart Room System

Let us assume that we want to make this architecture energy-efficient. One of the possible ways of doing this is using a self-adaptive system based on goals where the system performs an adaptation when a certain energy threshold is crossed. An even better mechanism (presented in this paper) will be to understand the correlation between the different parameters and creating a learning model that can forecast the possible energy consumption based on the current scenario, use the forecast to automatically decide the best adaptation strategy (using reinforcement learning) prior to crossing the threshold. The adaptation can be as simple as reducing the frequency of data transfer between components.

IV. The IoTArchML Approach

The approach conforms to the conceptual model of a self-adaptive system [16] [17]. Hence, it consists of two subsystems, namely the Managed System and Managing System.

The *Managed System* basically consists of the architecture of the system together with the set of QoS requirements that the system is supposed to meet. These consists of requirements on various QoS parameters such as energy, data traffic etc.

The *Managing System* consists of machine learning engine and adaptation manager. It is responsible for analyzing the data produced by the managed system and generates actionable insights in the form of adaptation decisions using the machine learning engine. The decisions generated by the machine learning engine are then verified by the adaptation manager which is responsible for executing the adaptation. This loop of learning and adaptation keeps continuing until an efficient architecture is generated with respect to the QoS requirements.

Figure 2 shows the *Machine Learning Engine* of the

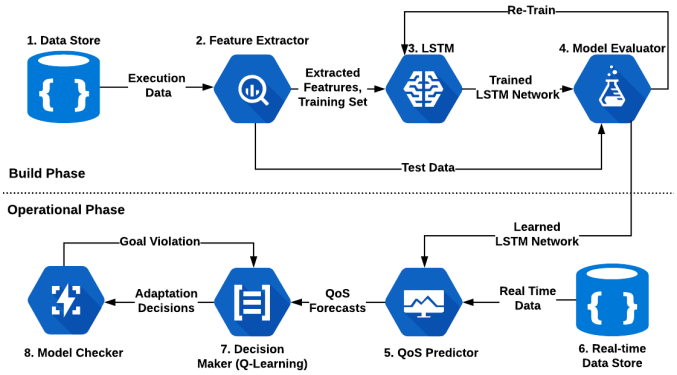

Fig. 2. Machine Learning Engine

IoTArchML approach. It internally consists of two phases: the *build* and the *operational* phase.

The *build* phase represents the process of building a machine learning model for the QoS parameter forecasting based on the historical data. On the other hand, the *operational* phase represents the process of performing QoS parameter predictions based on real-time data and using these predictions to generate the adaptation strategy.

A. Data Store

The first phase of the IoTArchML approach is to collect data. The data we require are logs containing information on every interaction that happens within and between the sensors at every instant of time. Such a data can be generated by either simulating (in case of a conceptual architecture) or executing (in case of an implemented architecture) the system architecture. This data is stored into the *Data Store* component, and further processed by the *Machine Learning Engine* for performing adaptation.

B. Data Cleaning and Feature Extraction

During the build phase, the data generated during the execution is first cleaned by the *Feature Extractor*. This process involves parsing the log files and structuring the data such that we get a multivariate time series data for every component in the architecture. For example, the log file contains information such as for each sensor component, for every instant of time, what was the data read, how many messages were sent, how many were received, etc. This data is further processed using techniques like Singular Value Decomposition (SVD) for identifying the co-related features and hidden dependencies that might exist among the features [18]. The *Feature Extractor* then divides the entire data set into a training set and testing set. The final set of features along with the training set is used to train a *Long Short-Term Memory Network (LSTM)* [19].

C. LSTM Network and Model Evaluator

LSTM Networks belongs to the class of Recurrent Neural Networks. LSTM's have the ability to handle the problem of long-term dependency better known in the literature as the *Vanishing Gradient Problem* [20], as compared to traditional RNN's. Due to this reason, they have shown to be very effective in time series forecasting [21]. LSTM networks as similar to a normal RNN's consists of three layers namely input layer, a hidden layer and output layer but unlike the normal RNN's the hidden layer of LSTM's is composed of recurrently connected subnets, called memory blocks which in turn consists of a combination of gates and cells. The multivariate time series data generated by the *Feature Extractor* is passed into the input layer of the LSTM network which is then passed through the memory blocks in the hidden layers. During the process of training, the LSTM's finds patterns in the data generated by different components of the architecture. It achieves this by identifying various types of co-relations among features that lead to different QoS values. This information is used to generate an LSTM network which can further forecast QoS parameters like energy consumption of a given component/system based on the data generated at the given instant of time, t.
The trained LSTM network is further evaluated using the test data generated by the feature extractor. The *Model Evaluator* checks the accuracy of the model on the test data. In the case of lower accuracy, retraining is performed and the trained LSTM network forms the *QoS Predictor*.

D. Real-time Data Store and QoS Predictor

During the operational phase, the data generated in real-time is ingested into the *Real-Time Data Store*. This data is then passed to the *Qos Predictor*. The QoS predictor then forecasts the QoS parameters for different intervals of time. These forecasts are then passed to the *Decision Maker*.

E. Decision Maker (Q-Learning) and Model Checker

Decision maker uses reinforcement learning techniques [22], in particular, *Q-Learning* [23] to decide on the adaptation strategy. Q-Learning is a widely used method for decision-making scenarios particularly because of their ability to come up with an optimal decision through state transition by performing actions that maximize the reward [24]. The state space consists of the set of possible states the system might enter into. These include states such as *high energy, high data traffic, high energy and high data traffic, etc*. The action space, on the other hand, consists of the possible adaptive decisions such as *increase frequency, reduce data exchange rate*. Together they form an $N \times M$ *Q-Table Matrix* where *N represents the states* and *M, the actions*. The Q-Table is initialized with 0 values. At a given instant of time, t, the QoS forecast produced by the *QoS Predictor* is used to determine the state inside the Q-Table. Q-Learning then selects an action (decision) from the action space. This can be performed in different ways and one of the commonly used strategies is a $\epsilon-$greedy strategy. It selects a greedy policy with probability 1-ϵ and selects a random action with probability ϵ. This action (decision) is then sent to the *Model Checker*. The *Model checker* verifies the decision by running a check to ensure that the overall system goals are not violated. If there is any violation, the model checker would return a negative value (-1) to the *Decision*

Maker which, is then considered as a penalty (negative reward) for the selected action (decision). If there is no violation then, the action (decision) becomes the adaptation strategy and the adaptation is executed. The QoS reduction/increase as predicted by the *QoS Predictor* at the time, $t+1$ is considered as the reward of the action (decision) selected by the Q-Learning at time t. In this manner, Q-Table is constantly updated using the Q-Function :

$$Q(s_t, a_t) = (1-\alpha)*Q(s_t, a_t)+\alpha*(r_t+\gamma*max(Q(s_t+1, a)))$$

where, $0 \leq \alpha \leq 1$ represents the learning rate, $0 \leq \gamma \leq 1$ represents the discount factor which can be considered as the weight given to the next action, r_t represents the reward for the action chosen at step t. For every decision taken, the Q-Table is updated based on the corresponding forecast. Knowing that q-learning converges over a period of time [25], the quality of the taken decision improves.

The use of machine learning techniques allows the IoTArchML to foresee the possible QoS violations, and proactively perform adaptation. This process of learning and adaptation keeps continuing thus enabling the IoT architecture to become efficient with respect to the given QoS requirements.

V. Conclusions and Future Work

In this paper, we came up with IoTArchML, an approach for performing proactive self-adaptation in IoT architectures based on machine learning techniques. However, further concrete realizations and validations are required to show the effectiveness and efficiency of the approach. Future work includes a concrete implementation of the approach to improve the energy efficiency of a given IoT architecture. This shall be performed by integrating the approach with the CAPS Framework [26]. The effectiveness of the approach shall be measured using standard measures like Root Mean Square Error (RMSE), Mean Average Precision (mAP) etc. and the efficiency of the approach shall be measured based on the quality of the adaptation decisions generated. The implementation shall also be extended to consider different QoS parameters like data traffic, throughput etc. Another interesting line of work shall be to investigate how decision making can be performed by using different trade-offs such as improving energy consumption by reducing data traffic or improving the utilization by compromising on energy consumption etc.

References

[1] Mazeiar Salehie and Ladan Tahvildari. Self-adaptive software: Landscape and research challenges. ACM Trans. Auton. Adapt. Syst., 4(2):14:1-14:42, May 2009. ISSN 1556-4665. doi: 10.1145/1516533.1516538. URL http://doi.acm.org/10.1145/ 1516533.1516538.

[2] A. Taivalsaari and T. Mikkonen. A roadmap to the programmable world: Software challenges in the iot era. IEEE Software, 34(1):72-80, Jan 2017. ISSN 0740-7459. doi: 10.1109/MS.2017.26.

[3] https://www.gartner.com/newsroom/id/3598917

[4] Gubbi, Jayavardhana, Rajkumar Buyya, Slaven Marusic, and Marimuthu Palaniswami. "Internet of Things (IoT): A vision, architectural elements, and future directions." Future generation computer systems 29, no. 7 (2013): 1645-1660.

[5] Thomas M. Mitchell. Machine Learning. McGraw-Hill, Inc., New York, NY, USA,1 edition, 1997. ISBN 0070428077, 9780070428072.

[6] Acosta Padilla, Francisco Javier. "Self-adaptation for Internet of things applications." PhD diss., Rennes 1, 2016.

[7] Federico Ciccozzi and Romina Spalazzese. Mde4iot: supporting the internet of things with model-driven engineering. In International Symposium on Intelligent and Distributed Computing, pages 67-76. Springer, 2016.

[8] Mirko D'Angelo, Annalisa Napolitano, and Mauro Caporuscio. Cyphef: a modeldriven engineering framework for self-adaptive cyber-physical systems. In Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings, pages 101-104. ACM, 2018.

[9] Fahed Alkhabbas, Romina Spalazzese, and Paul Davidsson. Architecting emergent configurations in the internet of things. In Software Architecture (ICSA), 2017 IEEE International Conference on, pages 221-224. IEEE, 2017.

[10] Nathalia Moraes do Nascimento and Carlos Jose Pereira de Lucena. Fiot: An agent-based framework for self-adaptive and self-organizing applications based on the internet of things. Information Sciences, 378:161-176, 2017

[11] Kim, Dongsun, and Sooyong Park. "Reinforcement learning-based dynamic adaptation planning method for architecture-based self-managed software." In Software Engineering for Adaptive and Self-Managing Systems, 2009. SEAMS'09. ICSE Workshop on, pp. 76-85. IEEE, 2009.

[12] Han Nguyen Ho and Eunseok Lee. Model-based reinforcement learning approach for planning in self-adaptive software system. In Proceedings of the 9th International Conference on Ubiquitous Information Management and Communication, page 103. ACM, 2015.

[13] Pawel Idziak and Siobhan Clarke. An analysis of decision-making techniques in dynamic, self-adaptive systems. In Self-Adaptive and Self-Organizing Systems Work- shops (SASOW), 2014 IEEE Eighth International Conference on, pages 137-143. IEEE, 2014.

[14] Tao Chen and Rami Bahsoon. Self-adaptive and online qos modeling for cloud-based software services. IEEE Transactions on Software Engineering, 43(5):453-475, 2017.

[15] Piergiuseppe Mallozzi. Combining machine-learning with invariants assurance techniques for autonomous systems. In Proceedings of the 39th International Conference on Software Engineering Companion, pages 485-486. IEEE Press, 2017.

[16] Kephart, J. O., Chess, D. M. (2003). The vision of autonomic computing. Computer, (1), 41-50.

[17] Danny Weyns. Software engineering of self-adaptive systems: an organised tour and future challenges. Chapter in Handbook of Software Engineering, 2017.

[18] Golub, Gene H., and Christian Reinsch. "Singular value decomposition and least squares solutions." Numerische mathematik 14.5 (1970): 403-420.

[19] Hochreiter, Sepp, and Jürgen Schmidhuber. "Long short-term memory." Neural computation 9, no. 8 (1997): 1735-1780.

[20] Hochreiter, S., Bengio, Y., Frasconi, P. and Schmidhuber, J., 2001. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies.

[21] Xingjian, S. H. I., Zhourong Chen, Hao Wang, Dit-Yan Yeung, Wai-Kin Wong, and Wang-chun Woo. "Convolutional LSTM network: A machine learning approach for precipitation nowcasting." In Advances in neural information processing systems, pp. 802-810. 2015.

[22] Sutton, Richard S., and Andrew G. Barto. Introduction to reinforcement learning. Vol. 135. Cambridge: MIT press, 1998.

[23] Watkins, Christopher JCH, and Peter Dayan. "Q-learning." Machine learning 8, no. 3-4 (1992): 279-292.

[24] Gosavi, Abhijit. "Reinforcement learning: A tutorial survey and recent advances." INFORMS Journal on Computing 21, no. 2 (2009): 178-192.

[25] Even-Dar, Eyal, and Yishay Mansour. "Learning rates for Q-learning." Journal of Machine Learning Research 5, no. Dec (2003): 1-25.

[26] Muccini, Henry, and Mohammad Sharaf. "Caps: Architecture description of situational aware cyber physical systems." In Software Architecture (ICSA), 2017 IEEE International Conference on, pp. 211-220. IEEE, 2017.

View publication stats